

就讀動機 (P)

一、從痛點解決到資訊工程的啟蒙

我對資訊科技的動機，始於解決開源社群中的繁簡轉換問題。在手動提交多次 PR 後，我意識到缺乏自動化工具的效率極低，因此自學開發了 1800 行的 Bash 工具 `replace.sh`，透過 `fd`、`rg`、`OpenCC` 與自製 `sed` 字典的分層架構，處理包含歧義詞判斷在內的繁簡轉換流程，並以此工具在 GitHub 上完成了約 150 個被合併的貢獻。這種「發現問題、實作工具、回饋社群」的循環，是我申請資訊工程學系最核心的驅動力。

二、實作的極限

高三階段，我開始利用 Docker 自架 Navidrome、Immich、AdGuard 等十餘項服務，並開發了支援多執行緒的 `yt.py` 下載器。但在架設過程中，我反覆碰到自己「能讓服務跑起來，卻說不清楚它為什麼能動」的窘境。例如設定 Caddy 反向代理時，光是搞懂 Caddyfile 的巢狀語法就耗費了大量時間；串接 CrowdSec 與 iptables 的防火牆規則時，我能照著文件把 bouncer 裝好，卻無法解釋封包從進入到被攔截的完整流程。容器之間無法互相連線時，我最後是靠切換成 host 網路模式硬解的，但對於 Docker 的 bridge network 為什麼不通、host 模式又犧牲了哪些隔離性，我說不出原因。這讓我意識到，缺乏網路與作業系統的基礎知識，我只能不斷地試錯與模仿，沒有能力從根本理解問題。

成大資工的大二必修「計算機組織」與「作業系統」能補足我在底層硬體與資源調度上的知識空白；「演算法」與「資料結構」則能將我目前直覺式的寫法轉化為具備複雜度分析思維的工程模式。這些不是錦上添花，是我繼續往前走的必要條件。

三、為何選擇成大資工

瀏覽系上 114 級專題成果後，我注意到莊坤達教授指導的多個分散式系統專題與我過去的經驗直接相關。其中一個專題解決了 Apache Airflow 在讀取大型日誌時的記憶體溢位問題，用 streaming interface 搭配 K-Way Merge Heap 取代整體排序，將記憶體用量從 3587MB 降至 33MB；另一個專題在 Kubernetes 環境中實作 DSCP 封包優先級控制器，讓關鍵應用的網路吞吐量提升了約 60 倍。這些專題處理的核心問題，與我在自架 Docker 服務時遭遇的困境本質相同，差別在於他們使用的是具備理論基礎的系統工程方法，而我仍停留於零星的修補。這個差距，正是成大資工系能為我補足的部分。

我計畫在學期間循此路徑，修習「電腦網路概論」與「作業系統」，並爭取加入分散式系統相關實驗室，將過去單打獨鬥的系統除錯經驗轉化為正規研究方法。期望畢業後能投入雲端資源調度領域，成為專注於基礎設施（**Infrastructure**）的分散式系統工程師，持續貢獻開源社群。

此外，張燕光教授指導的 **SpreadSketch** 多交換器架構 **Superspreader** 偵測、機器學習網路流量分析等專題，也展現了成大資工在網路安全與流量分析領域的研究深度。

智慧系統組的準備指引中明確要求提供 **GitHub** 專案連結與專案開發紀錄，這代表國立成功大學資訊工程系重視的不只是考試成績，而是學生是否具備從零開始解決真實問題的實作能力。成大作為南部最完整的資工研究重鎮，是我從課餘開發者成長為正規工程師最需要的環境。